WHAT IS GENERATIVE MODELLING?

Generative modelling is about learning how data is created

Instead of just learning to label or predict outcomes, a generative model tries to learn the underlying data distribution so it can create new data that looks realistic

A generative model learns “What does this data look like, statistically?”

Examples of data:

- Images (faces, objects)
- Text (sentences, articles)
- Audio (speech, music)
- Structured data (tables, sensor data)

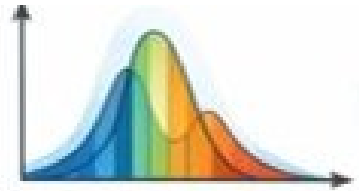


LEARNING A DATA DISTRIBUTION

- A data distribution describes:
  - What values are likely
  - How features relate to each other
  - What patterns commonly occur
- Example: In facial images,
  - Eyes are usually above the nose
  - Skin tones vary but are consistent across the face
  - Faces are symmetric
- A generative model learns these probabilistic patterns, not individual examples

Given training data:

$x_1, x_2, x_3, \ldots, x_n$



$P(x)$

learn data distribution



"What is the probability of seeing data point x?"

Once it knows this distribution, it can **sample new data** from it

PREDICTION, TRANSFORMATION, AND GENERATION

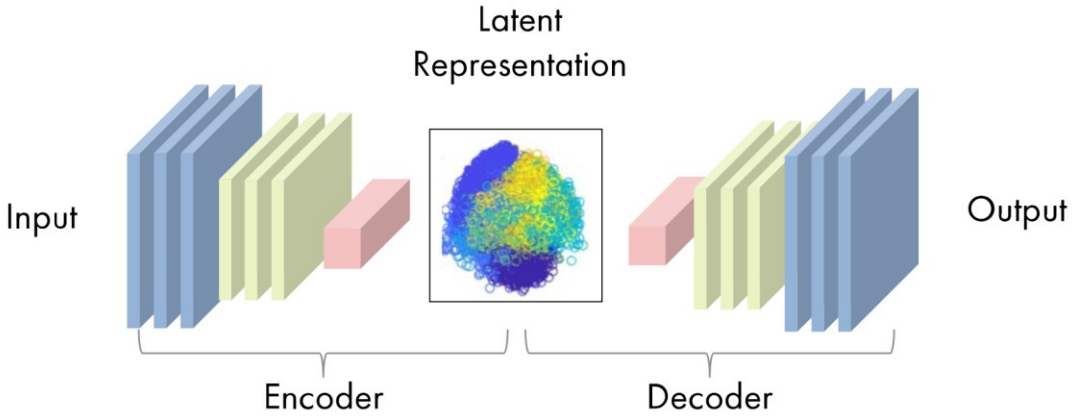
- **Prediction** means estimating an unknown value based on known input. In prediction, the model answers the question: *"Given x, what is y?"*
  - Prediction doesn't create new data, but outputs labels, numbers, or categories
  - Traditional ML algorithms perform prediction on input data
- **Transformation** means converting one form of data into another form. It changes *x* to *y*.
  - Transformation takes an input and produces a corresponding output, but doesn't create new data from scratch
  - Seq-to-seq models and autoencoders transform an input to a structurally-related output
- **Generation** is creating entirely new data points that follow the learned data distribution
  - Generated data is not explicitly present in the training corpus; it follows the same statistical properties as the original data
  - Transformers, diffusion models generate outputs using probabilistic modelling of data distributions

AUTOENCODERS

Encoder-decoder is a neural architecture consisting of an encoder that maps input to latent representation, and a decoder which maps the latent representation to output

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} y$$: *Here, x and y can be different*

Encoder-decoder architectures map input to a lower dimensional output



An **autoencoder** is a special case of an encoder-decoder where input and output are the same

The model is trained to compress and reconstruct its own input

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x}$$

Both the input and output are in the same data space

Reconstruction instead of prediction: same number of input and output nodes, but reduces output dimensions

*Encoder–decoder models learn **conditional mappings**; autoencoders learn **identity under compression***

Why autoencoders?
Reconstruction is the cheapest self-supervised way to test whether information survived the encoding

LATENT VARIABLES AND SPACES

Latent Variables

- Observed data is assumed to be generated from latent variables
- Latent variables capture underlying factors of variation rather than raw pixels
- The model learns a compact, abstract representation that explains the data
- Good latent variables separate structure from surface-level noise

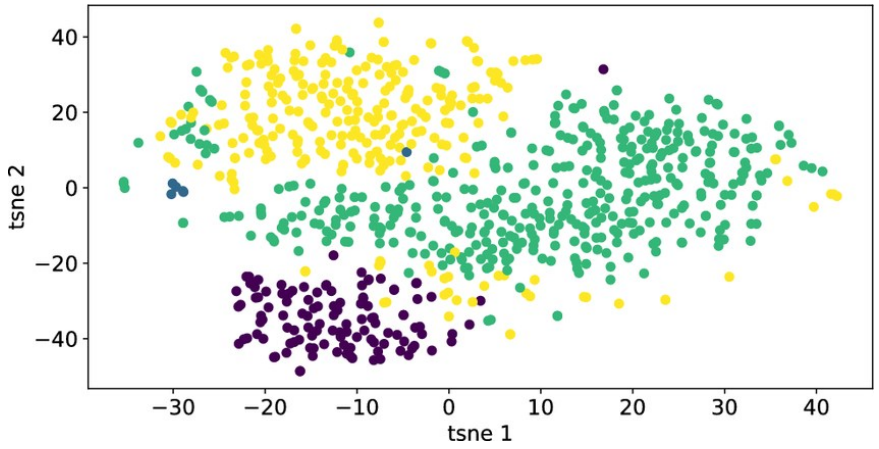
Latent Spaces

- A well-formed latent space is continuous and smooth
- Nearby latent points decode to similar outputs
- Interpolating between two latent points produces gradual, meaningful transitions
- Semantic properties change continuously, not abruptly

This continuity makes sampling and generation feasible,
since new latent points correspond to plausible data instances

LIMITATION - DETERMINISTIC LATENT SPACE

- For each input, the encoder outputs one fixed latent vector
- No notion of probability, uncertainty or multiple valid representations
- The reconstruction loss (e.g., MSE) is optimised directly
- So the model assumes one input to have only one correct output
  - Even for multiple valid outputs, the autoencoder collapses them into an average, giving blurry reconstructions
- The latent space for autoencoders is **deterministic** and **unstructured**

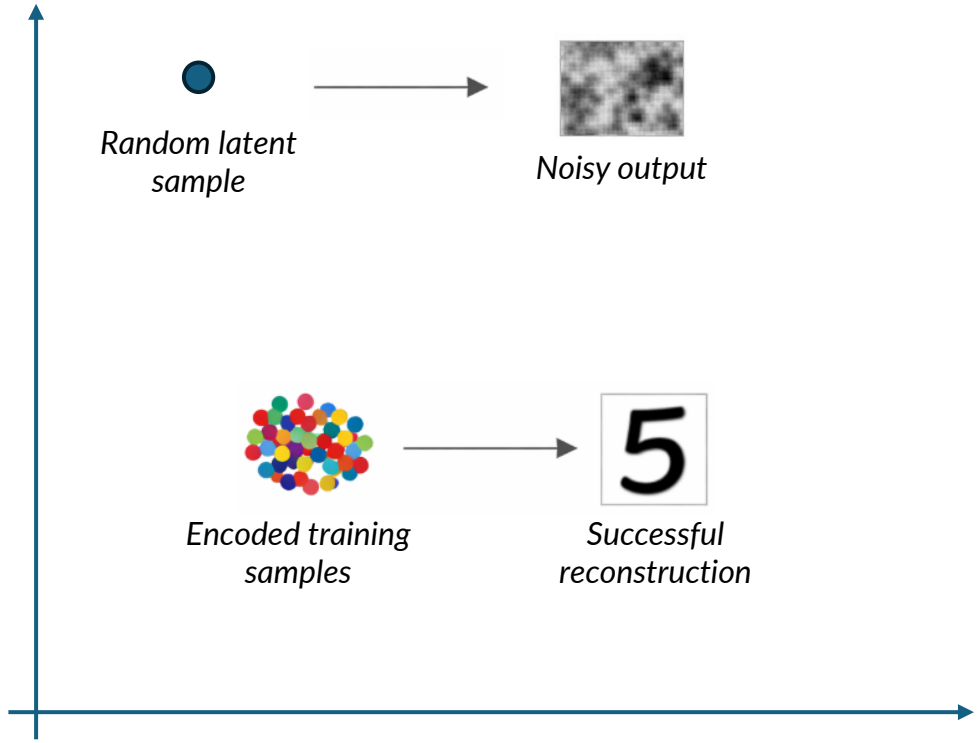


Points are sparse and irregular; No clear density or continuity

No distribution learned, nearby points are not guaranteed to decode to semantically similar outputs

RECONSTRUCTION AND GENERATION

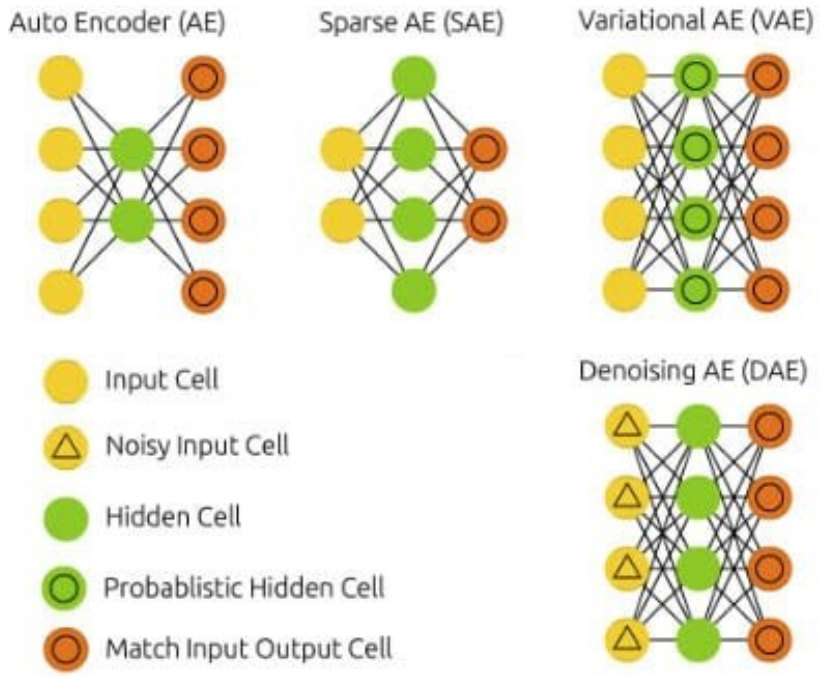
- Autoencoders are not generative models; reconstruction does not mean generation
- During training, the decoder is exposed **only** to latent vectors produced by the encoder
- These latent vectors correspond exactly to training samples
- And the random locations in the latent space remain unobserved during training
- Autoencoders only learn where the training samples land in the latent space, not where the valid samples can exist



When we sample random latent values to the decoder, it extrapolates beyond its learned domain, and gives noisy and distorted outputs

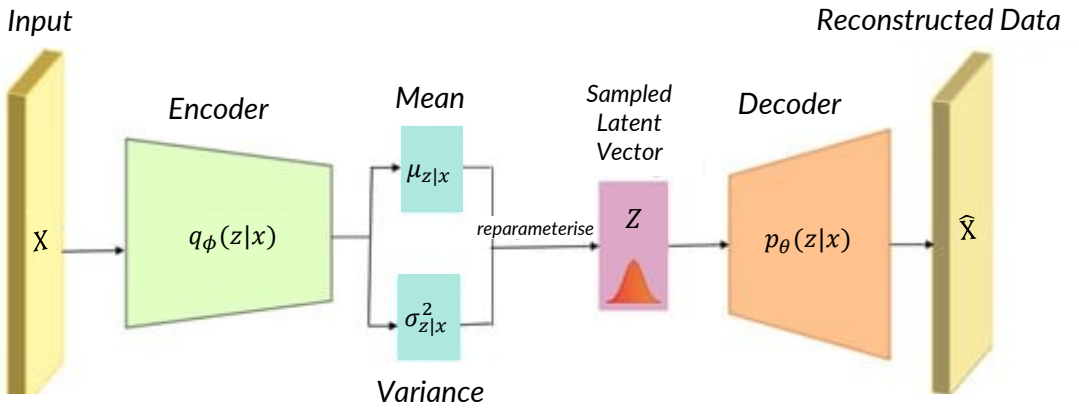
AUTOENCODERS FOR GENERATION

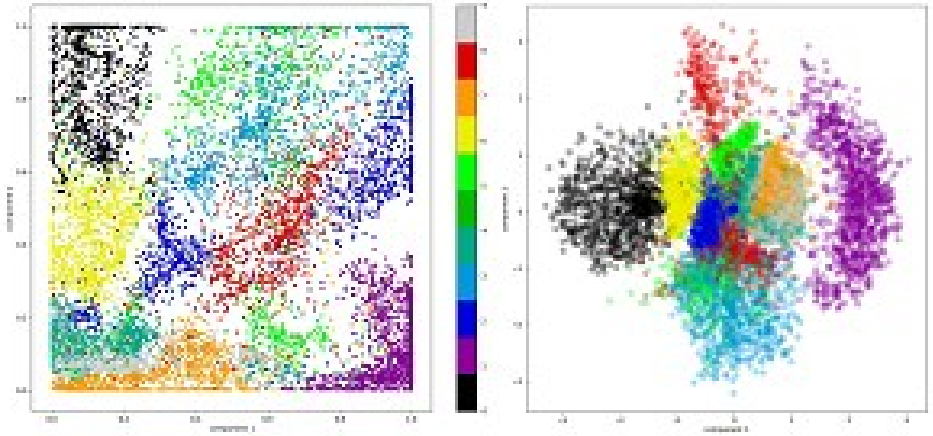
- One deterministic latent representation per input
- Standard autoencoders reconstruct well but cannot generate new, varied samples
- This motivates alternative autoencoder architectures such as:
  - Denoising Autoencoders
  - Sparse Autoencoders
  - Contractive Autoencoders
  - **Variational Autoencoders (VAEs)**
- To enable generation, the latent space must be structured and probabilistic
- From the above, VAEs explicitly learn a latent distribution, enabling sampling and controlled generation



VARIATIONAL AUTOENCODERS

- To generate, we need to model a distribution in latent space, not individual points
- Variational Autoencoder is a type of **generative autoencoder** that learns a **probability distribution** over latent variables (*variational inference*)
- Instead of encoding an input to a single point, the encoder learns a distribution in latent space
- Generation is done by sampling from this latent distribution and decoding it
- The variability acts to cover the entire range of input values to the decoder





A standard AE (left) can have huge gaps in possible input values to the decoder, but a VAE (right) has fewer input gaps

INTUITION BEHIND VAE

- Imagine every data point *(image, text, signal)* is created by first choosing a hidden cause z *(style, structure, intent)*, then generating the observable data x from that cause
- In a VAE:
  - z is not directly observed, we assume it comes from a simple distribution (e.g., a Gaussian)
  - x is produced by decoding z
- The standard autoencoder input has only one fixed point in latent space, making it sparse
- For VAE, the encoder outputs a distribution (mean + variance), and each input corresponds to a region, not a point
- This introduces controlled randomness and makes the space smooth and interpolatable

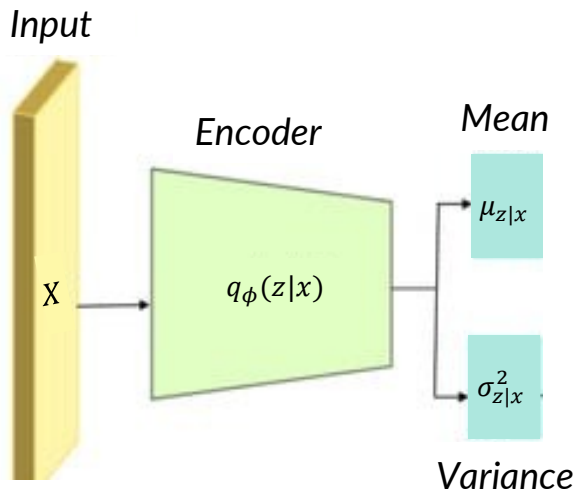


Why make z a random variable?

- Real data is noisy and ambiguous, many latent expressions can generate similar observations
- VAE learns how certain it should be about each encoding
  - Similar inputs may overlap in latent space
  - Uncertainty is explicitly represented

THE ENCODER

- The encoder takes in input data (like images) and learns its key features
- Unlike a regular encoder producing a single feature vector, the VAE encoder outputs the following two different vectors:
  - **Mean Vector**, $\mu(x)$: It represents the "center" of the features
  - **Variance Vector**, $\sigma^2(x)$: It represents the "spread" or uncertainty of those features; how much variance is there amongst the features
- These parameters define a probability distribution in the latent space, using which a slightly varied reconstructed data can be formed
- In other words, the encoder defines an *approximate posterior*, chosen to be a Gaussian distribution



LATENT SPACE SAMPLING

- VAE samples a point z from the probability distribution in latent space (mean and standard deviation vectors)
- Direct sampling breaks backpropagation

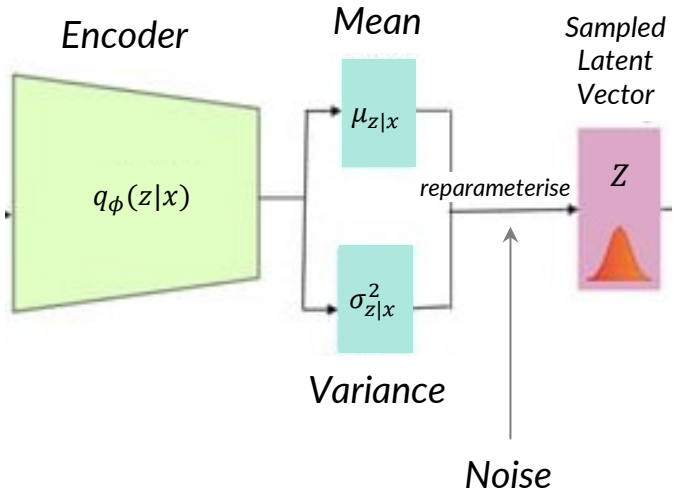
$$z \sim \mathcal{N}(\mu, \sigma^2)$$

- So VAEs use the **Reparameterisation Trick**, which rewrites sampling
- In reparameterisation, a small random noise is added during the sampling process:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- Randomness enables gradient flow and encoder becomes trainable despite stochastic sampling

z sampled from normal distribution with given mean and covariance



Note that we are adding std deviation here, not variance

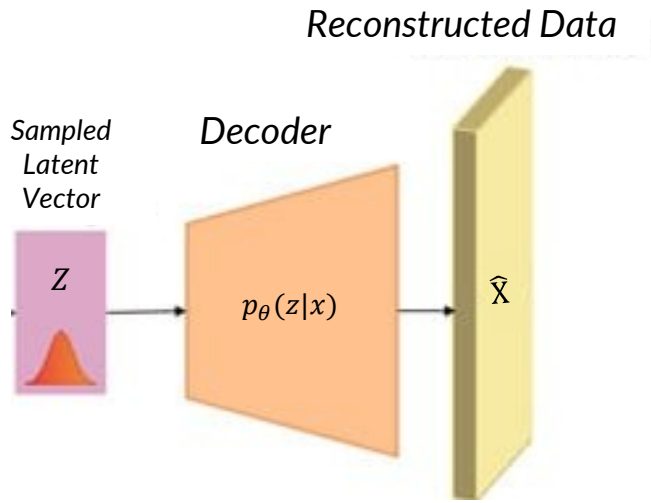
This is calculated from the variance vector $\sigma = \exp\left(\frac{1}{2}\log(\sigma^2)\right)$

The encoder also, technically, outputs log-variance vector $(\log(\sigma^2))$ *for easy computation*

THE DECODER

- The decoder takes the sampled point (z) and tries to reconstruct the original input (x)
- Because the latent space is forced to be a continuous distribution, the decoder learns that points "near" each other in the latent space should look similar
- In this way, the reconstructed data has a slight variation from the original data itself
- The decoder learns the *generative likelihood*, chosen to be a Gaussian distribution

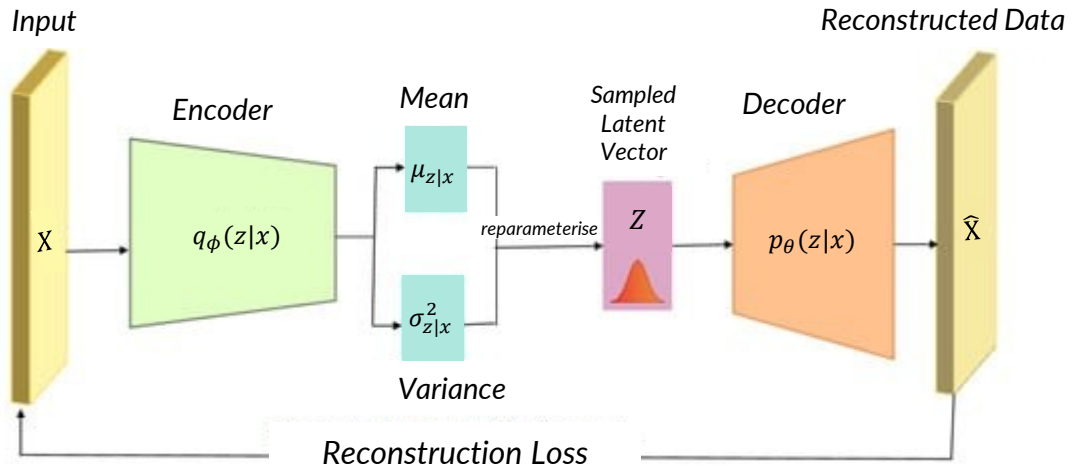
Encoder → μ, σ → sampling → decoder



RECONSTRUCTION LOSS

- Ensures that the output looks like the input
- We try to minimise the reconstruction loss
- Typically:
  - Binary Cross-Entropy (images in [0,1])
  - Mean Squared Error (continuous data)

$$\mathcal{L}_{\text{rec}} = E_{q(z|x)}[\log p(x|z)]$$



KULLBACK-LEIBLER (KL) DIVERGENCE

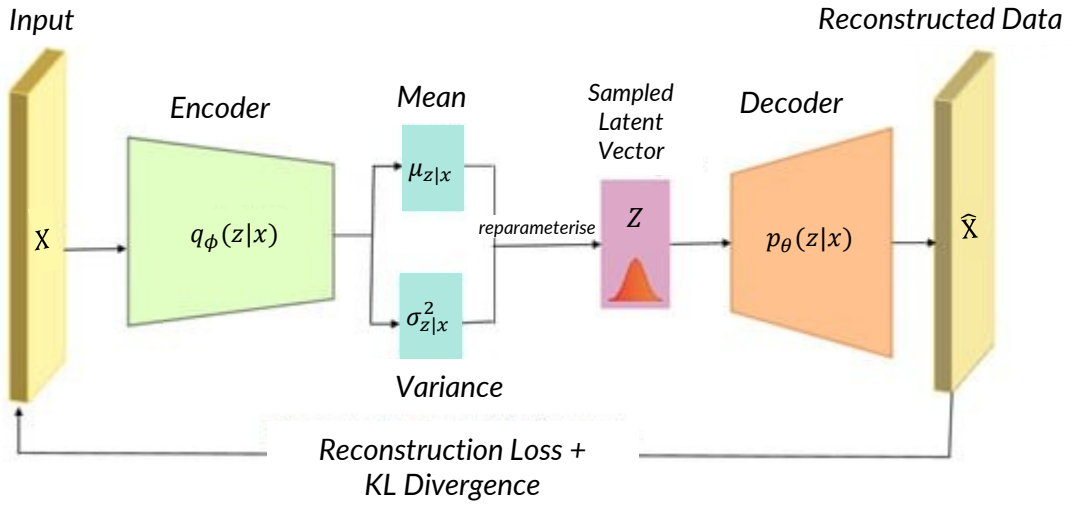
- Measures the deviation between two distributions
- Forces the latent space to stay organised and follow a standard normal distribution (learned posterior to match the prior)

$$\mathcal{L}_{KL} = D_{KL}\big(q(z \mid x) \parallel p(z)\big)$$

- For Gaussian distributions

$$\mathcal{L}_{KL} = \frac{1}{2}\sum(1 + \log\sigma^2 - \mu^2 - \sigma^2)$$

- Prevents the model from memorising the inputs – works as regularisation
- Latent space becomes continuous, dense, structured



VARIATIONAL INFERENCE

- VAE assumes that observed data x is generated from an unobserved latent variable z according to the hierarchical model

$$z \sim p(z), \qquad x \sim p(x \mid z),$$

  - $p(z)$ is a prior distribution over the latent space
  - $p(x \mid z)$ is a conditional likelihood parameterised by a neural network (the decoder)
- Generative process: Gaussian prior → decoder → data

- The objective is to learn the marginal data distribution (evidence), the distribution we want to model

$$p(x) = \int p(x, z)\, dz = \int p(x \mid z)\, p(z)\, dz$$

- The true posterior $p(z \mid x)$ is hard to compute, so VAEs introduce the *approximate posterior* $q_\phi(z \mid x)$, parameterised by the encoder network
- The *generative likelihood* $p_\theta(z \mid x)$ is learnt by the decoder

PROBABILITIES FOR VAE

Likelihood

$$p_\theta(\mathrm{x} \mid \mathrm{z})$$

- Describes how data is generated from a latent variable
- It is parameterised by the decoder network, and chosen based on the distribution type

Prior Distribution

$$p(z)$$

- This encodes assumptions about the latent variables before seeing any data
- In practice, the most common choice is $p(z) = \mathcal{N}(0, I)$

Marginal Data Distribution (evidence)

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$$

- The distribution we ultimately want to model, obtained by marginalising the latent variable

PROBABILITIES FOR VAE

True Posterior

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z)p(z)}{p_\theta(x)}$$

- Exact Bayesian posterior over latent variables, hard to compute as evidence is intractable

Approximate Posterior (encoder)

$$q_\phi(z \mid x)$$

- The variational distribution to approximate the true posterior, parameterised by the encoder network

As we already saw, we use the log of these probabilities for easier computations

TOTAL VAE OBJECTIVE (ELBO)

- The VAE is trained by maximising a lower bound on the data likelihood

$$\log p(x) \geq \mathcal{L}_{\text{VAE}}(x)$$

- The VAE objective becomes maximising this lower bound (ELBO)

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{KL}}$$

  - $\mathcal{L}_{\text{rec}}$ encourages accurate reconstruction
  - $\mathcal{L}_{\text{KL}}$ regularises the latent distribution

- The final expression becomes,

$$\mathcal{L}_{\text{VAE}}(x) = \mathbb{E}_{q(z|x)}[\log p(x \mid z)] - D_{\text{KL}}\big(q(z \mid x) \parallel p(z)\big)$$

- This balances reconstruction accuracy and latent space regularity
- This is why VAEs can be sampled, unlike autoencoders

WHY OUTPUTS FROM VAE ARE BLURRY?

- Decoder must produce valid outputs for all latent samples, not just one
- KL regularisation discourages sharp, isolated encodings
- The model averages over multiple plausible reconstructions
- Sharp details are sacrificed for distributional consistency
- VAEs trade visual sharpness for stability and coverage
- Models like GANs and diffusion remove or alter this trade-off to improve sample quality

